

Test Automation

Test automation is a prerequisite for unit testing and criteria-based testing.

One of the most widespread changes in software testing during the last decade has been the increased use of test automation. We introduced test automation in [Chapter 2](#) as implementing tests into executable test scripts. This chapter expands on that concept, starting with a complete definition.

Definition 3.9 Test automation : The use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions.

Software testing can be expensive and labor intensive, so an important goal of software testing is to automate as much as possible. Test automation not only reduces the cost of testing, it also reduces human error and makes regression testing easier by allowing a test to be run repeatedly with the push of a button.

Software engineers sometimes distinguish *revenue tasks*, which contribute directly to the solution of a problem, from *excise tasks*, which do not. For example, compiling a Java class is a classic excise task because, although necessary for the class to be executable, compilation contributes nothing to the behavior of that class. In contrast, determining which methods are appropriate to define a data abstraction in a Java class is a revenue task. Excise tasks are candidates for automation; revenue tasks usually are not. Software testing probably has more excise tasks than any other aspect of software development. Maintaining test scripts, rerunning tests, and comparing expected results with actual results are all common excise tasks that routinely use large amounts of test engineers' time.

Automating excise tasks serves the test engineer in many ways. First, eliminating excise tasks eliminates drudgery, thereby making the test engineer's job more satisfying. Second, automation frees up time to focus on the fun and challenging parts of testing, such as test design, a revenue task. Third, automation allows the same test to be run thousands of times without extra effort in environments where tests are run daily or even hourly. Fourth, automation can help eliminate errors of omission, such as failing to update all the relevant files with the new set of expected results. Fifth, automation eliminates some of the variance in test quality caused by differences in individual's abilities.

The rest of this chapter starts by exploring some of the things that make test automation hard (largely testability). It then breaks an executable test case down into components, and introduces one widely used test automation tool.

3.1 SOFTWARE TESTABILITY

Generally, software testability estimates how likely testing will reveal a fault if one exists. We are all familiar with software development projects where, despite extensive testing, faults continue to be found. Testability gets to the core of how easy or hard it is for faults to escape detection—even from well-designed tests.

Definition 3.10 Testability: The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met.

Testability is largely determined by two common practical problems; how to provide the test values to the software and how to observe results of test execution.

Definition 3.11 Software Observability: How easy it is to observe the behavior of a program in terms of its outputs, effects on the environment, and other hardware and software components.

Definition 3.12 Software Controllability: How easy it is to provide a program with the needed inputs, in terms of values, operations, and

behaviors.

These ideas are easily illustrated in the context of embedded software. Embedded software often does not produce output for human consumption, but affects the behavior of hardware. Thus, observability is quite low. Software for which all inputs are values entered from a keyboard is easy to control. But an embedded program that gets its inputs from hardware sensors is more difficult to control and some inputs may be difficult, dangerous, or impossible to supply (for example, how the automatic pilot behaves when a train jumps off-track). Many observability and controllability problems can be addressed with *simulation*, that is, by extra software built to “bypass” the hardware or software components that interfere with testing. Other types of software that often have low observability and controllability include component-based software, distributed software, and web applications.

Testability is crucial to test automation because test scripts need to control the execution of the component under test and to observe the results of the test. This discussion of test automation is a very short introduction. Many more details are available in the references given in the Bibliographic Notes. Several entire books are devoted to test automation.

3.2 COMPONENTS OF A TEST CASE

A test case is a multipart artifact with a definite structure. The following definitions are not standardized and the literature varies widely. The definitions are our own but are consistent with common usage. A test engineer must recognize that tests include more than just input values, but have many parts. The piece of a test case that is mentioned the most often contains what we call the test case values:

Definition 3.13 Test Case Values: The input values necessary to complete an execution of the software under test.

Note that the definition of test case values is quite broad. In a traditional batch environment, it is quite clear what a test case is. In a web application, a test case might generate part of a simple web page, or it might need to complete several commercial transactions. In a real-time system such as an avionics application, a test case might be so simple as to

be a single method invocation or as complex as an entire flight.

Test case values are inputs to the program that test designers use to directly satisfy the test requirements. They determine the quality of the testing. However, test case values are not enough. In addition to test case values, other inputs are often needed to run a test. These inputs may depend on the source of the tests, and may be commands, user inputs, or a software method with values for its parameters. To evaluate the results of a test, we must know what output a correct version of the program would produce for that test.

Depending on the software, the level of testing, and the source of the tests, the tester may need to supply other inputs to the software to affect controllability or observability. For example, if we are testing software for a mobile telephone, the test case values may be long distance phone numbers. We may also need to turn the phone on to put it in the appropriate state and then we may need to press “talk” and “end” buttons to view the results of the test case values and terminate the test. These ideas are formalized as follows.

Definition 3.14 Prefix Values: Inputs necessary to put the software into the appropriate state to receive the test case values.

Definition 3.15 Postfix Values: Inputs that need to be sent to the software after the test case values are sent.

Postfix values can be subdivided into two types.

Definition 3.16 Verification Values: Values necessary to see the results of the test case values.

Definition 3.17 Exit Values: Values or commands needed to terminate the program or otherwise return it to a stable state.

Once the execution terminates, a test case must determine whether the result of the test is valid, or is as expected. This is sometimes called the “test oracle” problem. A *test oracle* decides whether a test passed or failed. Thus, the results that the software should produce, if it behaves correctly, are included in the test case.

Definition 3.18 Expected Results: The result that should be produced

by the test case if the software behaves as expected.

A test case is the combination of all these components (test case values, prefix values, postfix values, and expected results). When it is clear from context, however, we will follow tradition and use the term “test case” in place of “test case values.”

Definition 3.19 Test Case: A test case is composed of the test case values, prefix values, postfix values, and expected results necessary for a complete execution and evaluation of the software under test.

We provide an explicit definition for a test set to emphasize that coverage is a property of a set of test cases, rather than a property of a single test case. You may sometimes see the term *test suite*, which usually means the same thing.

Definition 3.20 Test Set: A test set is a set of test cases.

The components in a test case are concrete realizations of the RIPR model from [Chapter 2](#). A test can be thought of as being designed to look for a fault in a particular location in the program. The prefix values are included to achieve reachability (*R*), the test case values to achieve infection (*I*), the postfix values to achieve propagation (*P*), and the expected results to reveal the failures (*R*). The expected results usually cannot include values for the entire output state of the program, so a well-designed test case should check the portion of the output state that is relevant to the input values and the purpose of the test.

As a concrete example, consider the function `estimateShipping()` that estimates shipping charges for preferred customers in an automated shopping cart application. Suppose we are writing tests to check whether the estimated shipping charges match the actual shipping charges. Prefix values, designed to reach (*R*) the `estimateShipping()` function in an appropriate state, might involve creating a shopping cart, adding various items to it, and obtaining a preferred customer object with an appropriate address. Test case values, designed to achieve infection (*I*), might be the type of shipping desired: overnight vs. regular. Postfix values, designed to achieve propagation (*P*) and make an infection result in an observable failure, might involve completing the order, so that actual shipping charges are computed. Finally, the revealing part (*R*) of the final order is probably implemented by extracting the actual shipping charge, although there are

many other parts of the final order that could also be incorrect.

Note that this test has an underlying complexity: we almost certainly do not want running the test to result in any merchandise leaving the warehouse or any customer receiving unordered goods. Solutions to this problem are presented in [Chapter 12](#).

Finally, wise test engineers automate as many test activities as possible. A crucial way to automate testing is to prepare the test inputs as executable tests for the software. This may be done as Unix shell scripts, input files, or through the use of a tool that can control the software or software component being tested. Ideally, the execution should be complete in the sense of running the software with the test case values, getting the results, comparing the results with the expected results, and preparing a clear report for the test engineer.

Definition 3.21 Executable Test Script: A test case that is prepared in a form to be executed automatically on the test software and produce a report.

The only time a test engineer would not want to automate is if the cost of automation outweighs the benefits. For example, this may happen if we are sure the test will only be used once or if the automation requires knowledge or skills that the test engineer does not have.

3.3 A TEST AUTOMATION FRAMEWORK

This book seldom refers to specific technologies or tools. Most of the knowledge we are trying to convey is not tied to a specific tool, and mentioning tools in textbooks invariably dates the book. The most notable exception is that all of our example programs are in Java; we had to pick some language and Java is convenient for several reasons. This section contains another exception. Although we try to present the concepts of test automation in a general way, we clarify the concepts with specific test automation examples. Although many test automation frameworks are available, we use JUnit because it is simple, widely used, includes features that represent all the ideas we want to present, and last but not least, is free. Many developers are moving to more sophisticated test automation technologies, but many are based on JUnit. In fact, the term “xUnit” is often used informally to mean a test framework based on, or similar to,

JUnit. Presenting JUnit early also gives instructors the opportunity to have students use JUnit in homework exercises¹.

We start by defining the term test framework in a general way.

Definition 3.22 Test Framework: A set of assumptions, concepts, and tools that support test automation.

A test framework provides a standard design for test scripts, and should include support for the test driver. A *test driver* runs a test set by executing the software repeatedly on each test. If the software component being tested is not standalone (that is, a method, class, or other component), then the test driver must supply the “main” method to run the software. The test driver should also compare the results of execution with the expected results (from the test case) and report the results to the tester.

The simplest form of driver is a `main()` method for a class. Effective programmers often include a `main()` for every class, containing statements that carry out simple testing of the class. For a typical class, the `main()` test driver will create some instances of the class, manipulate their values by calling mutator methods, and retrieve values for verification by calling observer methods. The driver can implement sophisticated techniques, such as those discussed in Part II of this book. This practice has evolved into the JUnit test framework, which provides a flexible collection of classes and API to develop test drivers. JUnit has, in turn, evolved into “*-Unit,” where similar functionality has been created for other languages and technologies.

Most test automation frameworks support:

- Assertions to evaluate expected results
- The ability to share common test data among tests
- Test sets to easily organize and run tests
- The ability to run tests from either a command line or a GUI

Most test automation frameworks are designed for unit and integration testing, although some specifically support system testing, and some are built to support testing over the web (HttpUnit, for example).

3.3.1 The JUnit Test Framework

JUnit scripts can be run as stand alone Java programs (from the command line) or within an integrated development environment (IDE) such as Eclipse. JUnit can be used to test an entire class, part of an object such as a method or some interacting methods, or interaction between several objects. That is, it is primarily used for unit and integration testing, not system testing.

JUnit embeds each test into one *test method*, and test methods are collected into *test classes*. Test classes include two parts:

1. A collection of test methods.
2. Methods to set up the program state before running each test (prefix values) and update the state after each test (postfix values).

Test classes are written using the methods in the *junit.framework.assert* class. Each test method checks a condition (*assertion*) and reports to the test runner whether the test failed or succeeded. Assertions are how expected results and the test oracle are encoded into JUnit tests. The test runner reports the result to the user. If in command line mode, the message is printed on screen. If in an IDE, the message is displayed in a window on the display. All assert methods return *void*. A few common methods are:

- *assertTrue (boolean)*: This is the simplest assertion, and, in principle, any assertion about program variables can ultimately be implemented using this assertion.
- *assertTrue (String, boolean)*: This assertion provides more information to the tester. If the assertion is true, the string is ignored. If the assertion is not true, the string is sent to the test engineer. It should provide a concise summary of the failure.
- *fail (String)*: This assertion puzzles many new test engineers, but it is extremely useful in situations where if a certain section of code is reached, that means the test has failed. As before, the string provides a summary to the test engineer. The `fail` method is often used to test exceptional behavior, although we also discuss another, often better, way in the `Min` class example.

The discussion in this section illustrates so-called “state-based” testing, where values produced by the unit under test are compared to known correct (“reference”) values. An important complement to state-based testing is “interaction-based testing,” where success is defined by how

objects communicate with each other. We discuss interaction-based testing in depth when we discuss test doubles in [Chapter 12](#).

JUnit uses the concept of a *test fixture*, which is the state of the test, as defined by the current values of key variables in the software under test. Test fixtures are especially useful when objects and variables are used by more than one test. The test fixture can be used to control the prefix values (initializations) and postfix values (reset values). This allows different tests to use the same objects without sharing state between tests. In other words, each test runs independently of other tests. Objects that will be used in test fixtures should be declared as instance variables in the JUnit class. They are initialized in a “@Before” method and reset or deallocated in an “@After” method.

[Figure 3.1](#) shows a very small class, `Calc`, and a JUnit test class, `CalcTest`. The method under test simply adds two integers. The JUnit test class has a single test with test values 2 and 3 and expected value 5. JUnit typically implements each test with a single `void` method without any parameters—`testAdd()` in this example. We will discuss other ways to implement tests later in the section. The annotation “@Test” defines a JUnit test and a common convention is to name test methods by prefixing the string “test” before the method name².

```
public class Calc
{
    static public int add (int a, int b)
    {
        return a + b;
    }
}

import org.junit.*;
import static org.junit.Assert.*;
public class CalcTest
{
    @Test public void testAdd()
    {
        assertEquals (5, Calc.add (2, 3));
    }
}
```

Figure 3.1. `Calc` class example and JUnit test.

[Figure 3.2](#) shows a more complex example that includes Java generics and tests for exceptions. Note that the `JavaDoc` documents exceptional returns as well as normal behavior. Naturally, these exceptions also need

to be tested, and the accompanying test class in [figures 3.3](#) and [3.4](#), `MinTest`, does exactly that. If `JavaDoc` comments are written well, testers can use them to write high-quality tests.

```
import java.util.*;
public class Min
{
    /**
     * Returns the minimum element in a list
     * @param list Comparable list of elements to search
     * @return the minimum element in the list
     * @throws NullPointerException (NPE) if list is null or
     *         if any list elements are null
     * @throws ClassCastException (CCE) if list elements are not mutually comparable
     * @throws IllegalArgumentException if list is empty
     */
    public static <T extends Comparable<? super T>> T min (List<? extends T> list)
    {
        if (list.size() == 0)
        {
            throw new IllegalArgumentException ("Min.min");
        }

        Iterator<? extends T> itr = list.iterator();
        T result = itr.next();

        if (result == null) throw new NullPointerException ("Min.min");

        while (itr.hasNext())
        {
            T comp = itr.next();
            if (comp.compareTo (result) < 0)
            { // throws NPE, CCE as needed
                result = comp;
            }
        }
        return result;
    }
}
```

Figure 3.2. Minimum element class.

```

import org.junit.*;
import static org.junit.Assert.*;
import java.util.*;

public class MinTest
{
    private List<String> list; // Test fixture

    @Before // Sets up - Called before every test method.
    public void setUp()
    {
        list = new ArrayList<String>();
    }

    @After // Tear down - Called after every test method.
    public void tearDown()
    {
        list = null; // Redundant in this example!
    }

    @Test
    public void testForNullList()
    {
        list = null;
        try {
            Min.min (list);
        } catch (NullPointerException e) {
            return;
        }
        fail ("NullPointerException expected");
    }

    @Test (expected = NullPointerException.class)
    public void testForNullElement()
    {
        list.add (null);
        list.add ("cat");
        Min.min (list);
    }

    @Test (expected = NullPointerException.class)
    public void testForSoloNullElement()
    {
        list.add (null);
        Min.min (list);
    }
}

```

Figure 3.3. First three JUnit tests for `Min` class.

```

@Test (expected = ClassCastException.class)
@SuppressWarnings ("unchecked")
public void testMutuallyIncomparable()
{
    List list = new ArrayList();
    list.add ("cat");
    list.add ("dog");
    list.add (1);
    Min.min (list);
}

@Test (expected = IllegalArgumentException.class)
public void testEmptyList()
{
    Min.min (list);
}

@Test
public void testSingleElement()
{
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue ("Single Element List", obj.equals ("cat"));
}

@Test
public void testDoubleElement()
{
    list.add ("dog");
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue ("Double Element List", obj.equals ("cat"));
}
}

```

Figure 3.4. Remaining JUnit test methods for `Min` class.

Class `MinTest` is split across two pages because of its length. The test fixture methods and the first three test methods are shown in [Figure 3.3](#). The “`@Before`” method encodes the prefix part of the test. It puts the test object into a proper initial state by creating a new `List` object. The “`@After`” method encodes the postfix part of the test. It resets the state of the test object by setting the object reference to null. Strictly speaking, the `@After` method is redundant since the `@Before` method resets the reference anyway, but good engineering practice is to be conservative: “measure twice, cut once.”

[Figure 3.3](#) also shows three separate tests where the expected result is `NullPointerException`. The JavaDoc specification indicated that

`NullPointerException` should be thrown if either the list is `null` or if any element in the list is `null`, thus we need an explicit test for each situation.

The first test illustrates the JUnit `fail` statement. For this test to pass, we expect to throw, and then catch, a `NullPointerException`. If no exception is thrown, or if a different exception is thrown³, the `fail` statement is reached, and the test correctly reports failure. Again, no `assert` statement is needed.

The second test illustrates an alternate approach to testing exceptional behavior. Specifically, the `@Test` annotation can be augmented with the class of the specific exception expected. This second approach is usually more straightforward to program and understand. Also, by identifying the expected exception by class it avoids some common mistakes that arise due to the inheritance structure of the Java exception classes. Hence, we recommend implementing test cases for exceptional returns with this second approach. Note that no `assert` or `fail` statements are needed.

The reason for the third `NullPointerException` test is more subtle. Even good programmers might overlook the possibility of a list that contains only a single `null` element and nothing else. Indeed, it is this test that forces the `Min` method to include an explicit `NullPointerException` throw after the variable `result` is initialized. To fully understand why this is needed, we suggest commenting out this line of code and rerunning the test set. To test this situation, we include an additional test to cover a single `null` element.

Figure 3.4 shows four additional tests for the `Min` class. The first two tests are for exceptions. Note that despite the use of generics, it is possible to call `Min` with a list of elements that are not mutually comparable—or even with elements that do not implement the `Comparable` interface at all. The reasons for this are subtle and complex⁴, but the message for the test engineer is very simple: if you think it might be possible, you should test it! Notice that this test requires “raw” types, about which the Java compiler duly warns us. Following good Java practice, we use the “`@SuppressWarnings`” annotation to suppress this warning.

The final two tests in the `MinTest` class address “normal” behavior. The balance between exceptional returns and normal returns shown in this example (five to two) is hardly unusual. Exceptional behavior is notoriously harder to program correctly than “happy path” behavior,

Unfortunately, many inexperienced testers (and programmers) will focus primarily on testing expected behavior, and test few, if any, exceptional conditions. When evaluating tests, one of the first things to check is how thoroughly exceptional behavior is considered.

3.3.2 Data-Driven Tests

Sometimes, the same test method needs to be run multiple times, with the only difference being the input values and the expected output. For example, the `add()` method in the `Calc` class should be tested with several values and expected sums. Repeatedly cutting and pasting the same test method and subsequently editing the inputs and expected outputs results in completely unmaintainable test code (and lots of chances for mistakes). A better solution is to write the test once and then supply the data values in a table. This approach is commonly called *data-driven testing*. The JUnit `Parameterized` mechanism implements data-driven testing. We avoid the term “parameterized” as much as possible in this discussion because it is overloaded in the context of unit testing with different, and conflicting, definitions.

[Figure 3.5](#) shows `DataDrivenCalcTest`, a Java class that defines data-driven JUnit tests for the `Calc` class. The import statements at the beginning of the file bring in the JUnit classes needed.

```

import org.junit.*;
import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;
import org.junit.runners.Parameterized.Parameters;
import static org.junit.Assert.*;
import java.util.*;

@RunWith (Parameterized.class)
public class DataDrivenCalcTest
{
    public int a, b, sum;

    public DataDrivenCalcTest (int a, int b, int sum)
    { // Note Constructor
        this.a = a;
        this.b = b;
        this.sum = sum;
    }

    @Parameters
    public static Collection<Object[]> calcValues()
    {
        return Arrays.asList (new Object [][] {{1, 1, 2}, {2, 3, 5}});
    }

    @Test
    public void additionTest()
    {
        assertTrue ("Addition Test", sum == Calc.add (a,b));
    }
}

```

Figure 3.5. Data-driven test class for `Calc`.

JUnit uses the Java class mechanism to implement data-driven tests. Specifically, the table of inputs and expected outputs come from a user-defined method annotated as `@Parameters`. This method returns a data-driven table in the form of a collection of `Object` arrays. JUnit expects the number of objects in each array to correspond to the number of formal arguments in the constructor for the JUnit test class. In this example, the number of objects is three: two input values (addends) and the expected result (the addends' sum). This matches the number of arguments in the constructor `DataDrivenCalcTest()`.

JUnit creates a new instance of the test class for each array in the collection returned by the `@Parameters` method. Methods in the test class use the instance variables initialized in the constructor call to test behavior in the same way as normal test methods. In the example, the

@Parameters method `calcValues()` returns a collection with two arrays of inputs and expected outputs, and hence JUnit calls the constructor `DataDrivenCalcTest()` twice. The arguments in the first constructor call come from the first array returned by `calcValues()`, and the arguments in the second constructor call come from the second array returned by `calcValues()`⁵. For each of the two resulting `DataDrivenCalcTest` objects, JUnit executes the test method `additionTest()`.

3.3.3 Adding Parameters to Unit Tests

None of the test methods discussed so far have had explicit parameters. Allowing the use of parameters in test methods is extremely powerful—both theoretically and practically. The JUnit Theory mechanism allows test engineers to define test methods with parameters.

Sidebar

JUnit Theories: Universal Quantification for Testing

Consider the universally quantified assertion:

$$\forall x \in X \bullet P(x) \rightarrow Q(x)$$

This assertion can be interpreted to mean that, for all values in a particular domain X , if the precondition P is true, then the postcondition Q is also true.

The normal approach to such an assertion is using a mathematical proof to show that the assertion is, indeed, a theorem. Testing is usually considered ill-suited to showing such an assertion—primarily because the domain of interest, represented by X in this example, is often very large, and hence X cannot be enumerated exhaustively.

Unit tests with parameters explore a middle ground between mathematical proof and ordinary testing. They promise to use the practical power of testing, at least partially, to demonstrate the validity of universally quantified assertions. From a testing perspective, this is radical!

A test engineer writing a test method with parameters proceeds

mathematically. She hypothesizes that for all possible combinations of parameters that satisfy the preconditions, the postcondition is also true for whatever action the test implements. Those familiar with design-by-contract might recognize this important pattern: Precondition, Action, Postcondition.

Of course, there is no way to try all possible values, or else our tests will never finish running. But the test engineer should not be concerned with where values come from, or how many there are, when specifying the theory itself. Those concerns, which are addressed differently in different approaches to test methods with parameters, can wait. Instead, she should focus on writing a valid test method—and leave it up to the test engine to find a counterexample if possible.

Figure 3.6 shows an example JUnit theory about sets of strings. The theory is implemented in a method annotated with `@Theory`. Notice that this method has two parameters, a set of strings and a string. The “Action” part of theory, implemented in ordinary Java, removes a string from a set and then adds the string back in. The postcondition of the theory, implemented by the `assertTrue` statement, asserts that the resulting set is the same as the starting set. Of course, this theory is only true if the starting set already contains the string being removed. In other words, the theory has a precondition. This precondition, implemented in the `assumeTrue` statement, states that the starting set contains the necessary string. The theory also has a precondition that starting set not be `null`; otherwise, the theory will fail in an uninteresting manner via `NullPointerException`.

```
@Theory
public void removeThenAddDoesNotChangeSet
    (Set<String> someSet, String str) // Parameters!
{
    assumeTrue (someSet != null);      // Precondition
    assumeTrue (someSet.contains (str)); // Precondition
    Set<String> copy = new HashSet<String>(someSet); // Action
    copy.remove (str);
    copy.add (str);
    assertTrue (someSet.equals (copy)); // Postcondition
}
```

Figure 3.6. JUnit Theory about sets.

So far, this example leaves out an important question: what values should be substituted for the parameters in the test? Put another way, the parameters in the JUnit `Theory` method provides a “box” to hold test inputs. It is upto the test engineer and the test driver framework to decide what values to put into the box. [Figure 3.7](#) shows the JUnit approach to supply parameters to the set example.

```
@DataPoints
public static String[] animals = {"ant", "bat", "cat"};

@DataPoints
public static Set[] animalSets = {
    new HashSet (Arrays.asList ("ant", "bat")),
    new HashSet (Arrays.asList ("bat", "cat", "dog", "elk")),
    new HashSet (Arrays.asList ("Snap", "Crackle", "Pop"))
};
```

Figure 3.7. JUnit Theory data values.

In JUnit, possible values for parameters are explicitly listed in `@DataPoints` objects, which are arrays of Java data types. JUnit matches data values to parameters by type: if the data value has the same type as the parameter, then JUnit plugs it in ⁶. The number of tests is the cross-product of all the possible values for each parameter in the test. For example, the test in [Figure 3.6](#) has two parameters, one of type `String` and the other of type `Set`. [Figure 3.7](#) has three values of type `String` and three values of type `Set`, thus, this example has $3 * 3 = 9$ possible combinations of values, four of which satisfy the precondition. All four combinations that satisfy the precondition also satisfy the postcondition—exactly what we expect from a valid theory. For the five combinations that do not satisfy the precondition, JUnit does not evaluate the postcondition. If a precondition is not satisfied, then the postcondition does not apply.

Data-driven testing can suffer from a combinatorial explosion in the number of tests. Remember that the number of potential tests is given by the cross-product of the possible values for each of the parameters in the unit test. For small sets of data values, the number of tests is not usually a problem. However, for large sets of data values, or for test methods with many parameters, the number of actual tests may balloon quite quickly. Testers must be aware of this and adjust accordingly if the test framework generates more tests than can be practically (or helpfully) run.

3.3.4 JUnit from the Command Line

The above examples are enough to run JUnit inside an IDE. To run from a command line, however, a main method is needed. [Figure 3.8](#) shows the additional class needed to run the tests for the `Min` class. If a test fails, JUnit gives the location of the failure and any exceptions that were thrown. If JUnit is run from the command line, the format of the feedback is simply the normal stack trace that the Java runtime system supplies for any uncaught exception. IDEs tend to format the feedback more clearly.

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import junit.framework.JUnit4TestAdapter;

// This section declares all of the test classes in the program.
@RunWith (Suite.class)
// Add more test classes by inserting a .class file name
// inside the curly brackets, separate by commas.
@Suite.SuiteClasses ({ MinTest.class })

public class AllTests
{
    // Execution begins in main(). This test class executes a
    // test runner that tells the tester if any tests fail.
    public static void main (String[] args)
    {
        junit.textui.TestRunner.run (suite());
    }

    // The suite() method helps when using JUnit 3 Test Runners or Ant.
    public static junit.framework.Test suite()
    {
        return new JUnit4TestAdapter (AllTests.class);
    }
}
```

Figure 3.8. AllTests for the `Min` class example.

3.4 BEYOND TEST AUTOMATION

Test practitioners widely agree that test automation is an essential way to make testing more efficient and effective. Test automation frameworks, however, are not “silver bullets.” They do not solve the core technical problem of software testing: **What test values to use?** This is the subject of test design. After test driven development in [Chapter 4](#), [Chapter 5](#)

discusses criteria-based test design in general, then the next four chapters give specific test criteria for designing tests.

EXERCISES

Chapter 3.

1. Why do testers automate tests? What are the limitations of automation?
2. Give a one-to-two paragraph explanation for how the **inheritance** hierarchy can affect controllability and observability.
3. Develop JUnit tests for the `BoundedQueue` class. A compilable version is available on the book website in the file `BoundedQueue.java`. Make sure your tests check every method, but we will not evaluate the quality of your test designs and do not expect you to satisfy any test criteria. Turn in a printout of your JUnit tests and either a printout or a screen shot showing the results of each test.
4. Delete the explicit *throw* of `NullPointerException` in the `Min` program ([Figure 3.2](#)). Verify that the JUnit test for a list with a single *null* element now fails.
5. The following JUnit test method for the `sort()` method has a non-syntactic flaw. Find the flaw and describe it in terms of the RIPR model. Be as precise, specific, and concise as you can. For full credit, you must use the terminology introduced in the book.
In the test method, `names` is an instance of an object that stores strings and has methods `add()`, `sort()`, and `getFirst()`, which do exactly what you would expect from their names. You can assume that the object `names` has been properly instantiated and the `add()` and `sort()` methods have already been tested and work correctly.

```

@Test
public void testSort()
{
    names.add ("Laura");
    names.add ("Han");
    names.add ("Alex");
    names.add ("Ashley");
    names.sort();
    assertTrue ("Sort method", names.getFirst().equals ("Alex"));
}

```

6. Consider the following example class. `PrimeNumbers` has three methods. The first, `computePrimes()`, takes one integer input and computes that many prime numbers. `iterator()` returns an `Iterator` that will iterate through the primes, and `toString()` returns a string representation.

```

public class PrimeNumbers implements Iterable<Integer>
{
    private List<Integer> primes = new ArrayList<Integer>();

    public void computePrimes (int n)
    {
        int count = 1; // count of primes
        int number = 2; // number tested for primeness
        boolean isPrime; // is this number a prime
        while (count <= n)
        {
            isPrime = true;
            for (int divisor = 2; divisor <= number / 2; divisor++)
            {
                if (number % divisor == 0)
                {
                    isPrime = false;
                    break; // for loop
                }
            }
            if (isPrime && (number % 10 != 9)) // FAULT
            {
                primes.add (number);
                count++;
            }
            number++;
        }
    }

    @Override public Iterator<Integer> iterator()
    {
        return primes.iterator();
    }
}

```

```

@Override public String toString()
{
    return primes.toString();
}
}

```

`computePrimes()` has a fault that causes it **not** to include prime numbers whose last digit is 9 (for example, it omits 19, 29, 59, 79, 89, 109, . . .). If possible, describe five tests. You can describe the tests as sequences of calls to the above methods, or briefly describe them in words. Note that the last two tests require the test oracle to be described.

- (a) A test that does not reach the fault
- (b) A test that reaches the fault, but does not infect
- (c) A test that infects the state, but does not propagate
- (d) A test that propagates, but does not reveal
- (e) A test that reveals the fault

If a test cannot be created, explain why.

7. Reconsider the `PrimeNumbers` class from the previous exercise. Normally, this problem is solved with the Sieve of Eratosthenes [Wikipedia, [2015](#)]. The change in algorithm changes the consequences of the fault. Specifically, false positives are now possible in addition to false negatives. Recode the algorithm to use the Sieve approach, but leave the fault. What is the first false positive, and how many “primes” must a test case generate before encountering it? What does this exercise show about the RIPR model?
8. Develop a set of data-driven JUnit tests for the `Min` program. These tests should be for normal, not exceptional, returns. Make your `@Parameters` method produce both *String* and *Integer* values.
9. When overriding the `equals()` method, programmers are also required to override the `hashCode()` method; otherwise clients cannot store instances of these objects in common `Collection` structures such as `HashSet`. For example, the `Point` class from [Chapter 1](#) is defective in this regard.
 - (a) Demonstrate the problem with `Point` using a `HashSet`.
 - (b) Write down the mathematical relationship required between `equals()` and `hashCode()`.

- (c) Write a simple JUnit test to show that `Point` objects do not enjoy this property.
 - (d) Repair the `Point` class to fix the fault.
 - (e) Rewrite your JUnit *test* as an appropriate JUnit *theory*. Evaluate it with suitable `DataPoints`.
10. Replace each occurrence of a set with a list in the JUnit theory `removeThenAddDoesNotChangeSet`. Is the resulting theory valid or invalid? How many of the tests that pass the precondition also pass the postcondition? Explain.

3.5 BIBLIOGRAPHIC NOTES

Our definition of test automation was adapted from Dustin et al. [Dustin et al., 1999], which goes into great detail on the practical and problematic aspects of test automation. The descriptions of excise and revenue tasks were taken from Cooper [Cooper, 1995].

Several different definitions of testability have been published. According to the 1990 IEEE standard glossary [IEEE, 2008], testability is the “degree to which a component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met.” Voas and Miller [Voas and Miller, 1995] defined software testability by focusing on the “probability that a piece of software will fail on its next execution during testing if the software includes a fault.” Binder [Binder, 1994, Binder, 2000] defined testability in term of controllability and observability. Controllability is the probability that users are able to control a component’s inputs (and internal state). Observability is the ability that users have to observe a component’s outputs. If users cannot control the inputs, they cannot be sure what caused the output. If users cannot observe the output of a component under test, they cannot be sure if the execution was correct. Freedman [Freedman, 1991] also described testability based on the notions of observability and controllability. In his terms, observability captures the degree to which a component can be observed to generate the correct output for a given input, and controllability refers to the ease of producing all values of its specified output domain.

Our definition of testability is adapted from the IEEE standards [IEEE, 2008]. Our definitions for observability and controllability are adapted

from Freedman [Freedman, 1991].

The observation that information hiding reduces controllability, thereby making testing harder, is due to Voas [Voas, 1992].

The multiple parts of the test case are based on research in test case specifications by Balcer and Stocks [Balcer et al., 1989, Stocks and Carrington, 1993].

JUnit is simply a test driver. Readers skeptical of JUnit’s simplicity might wish to review the half-page implementation of basic JUnit provided by Bloch, page 171 [Bloch, 2008]. The power of JUnit is in its uniformity: it encourages all Java programmers write tests the same way. Buest and Suileman [Beust and Suleiman, 2008] describe TestNG, a successor to JUnit that includes features useful for testing large scale projects. TestNG uses the term *data-driven testing* consistently with our usage, but TestNG includes a richer support framework. Tillmann and Schulte [Tillmann and Schulte, 2005] developed the approach of adding parameters to unit tests. The Pex test generation framework [Tillmann and de Halleux, 2008] automatically identifies possible values to substitute in for these parameters.

¹ At George Mason, we introduce JUnit in our second semester programming class and the first graduate class in the software engineering MS program. Of course, we also use JUnit in our testing classes.

² JUnit 3 required the convention of starting test methods with the string `test`. The annotations used in JUnit 4 give the compiler a chance to catch mistakes that would otherwise be silently ignored.

³ To be precise, since the Java exception mechanism uses the type hierarchy, the `catch` block in this example intercepts any subclass of `NullPointerException`.

⁴ Java generics are implemented by *erasure* so as to be backwards compatible with older versions of Java. Put another way, Javagenerics are only analyzed by the compiler; there is no trace of them left in the Java bytecode.

Java generics allow the programmer to write code with better type safety. In practical terms, this means that many potential sources of `ClassCastException` are turned into compile-time errors. This is a good thing! It is *always* better to identify a problem at compile time than to wait for a failing test case, or, worse, a field failure.

Unfortunately, Java generics can guarantee type safety only if *all* Java code in a system uses generics instead of so-called “raw” types. Generally, the test

engineer will not be in a position to ensure that raw types have been eliminated. Bottomline: type-safety violations that result in *ClassCastException* are possible even in code that properly uses generics. Hence, it is often necessary to write tests for type-safety violations in addition to using the Java generics mechanism.

- 5 Since JUnit uses Java reflection to implement these calls, the compiler cannot check that the number and type of objects in the array returned by the `@Parameters` method match the number and type of objects expected by the test class constructor.
- 6 To be precise, JUnit uses the Java `instanceof` test to determine whether a given object can be associated with a particular parameter. JUnit is implemented with the Java reflection mechanism, which is a runtime facility. Hence, JUnit *cannot* take advantage of Java generics for type matches, and instead relies exclusively on “raw” types.